

Questions 21–40 — Full Solutions

Recursion · Hashing · Sorting · Arrays start · In Depth · C++ & Python

What's inside. Items 21–40 of the sheet, each solved in **both C++ and Python** with the approach, full code, and complexity. This block is where DSA really starts: the **recursion patterns** (21–28), **hashing** for O(1) lookups (29–31), the **four core sorts + their recursive versions** (32–38), and your first two **array** problems (39–40). All code was compiled with `g++ -std=c++17` and run in Python, and the sorts were checked against 300 random arrays each.

Read the Approach first, write both versions yourself, then compare. These are the patterns you'll reuse for the next 400 problems.

Recursion (Q21–Q28)

Q21. Print Name N Times

Easy

■ **Approach:** Base case: $n==0$, stop. Otherwise print once and recurse with $n-1$. Each call shrinks n toward the base case.

C++

```
void printName(string s, int n){
    if(n == 0) return;      // base case
    cout << s << "\n";    // work
    printName(s, n - 1);    // smaller
}
```

PYTHON

```
def print_name(s, n):
    if n == 0:              # base case
        return
    print(s)                # work
    print_name(s, n - 1)    # smaller
```

Complexity & notes: $O(n)$ time, $O(n)$ stack. The base-case + smaller-call template underlies every recursion below.

Q22. Print 1 to N (recursion)

Easy

■ **Approach:** To print in INCREASING order, recurse FIRST (down to 1), then print on the way back up. Printing after the recursive call reverses the natural top-down order.

C++

```
void oneToN(int n){
    if(n == 0) return;
    oneToN(n - 1);          // go deep first
    cout << n << " ";      // print on the way back
}
```

PYTHON

```
def one_to_n(n):
    if n == 0:
        return
    one_to_n(n - 1)         # go deep first
    print(n, end=" ")       # print on the way back
```

Complexity & notes: $O(n)$. Key idea: WHERE you print (before vs after the recursive call) flips the order. Print-after gives 1..N.

Q23. Print N to 1 (recursion)

Easy

■ **Approach:** For DECREASING order, print FIRST, then recurse. The mirror of Q22 — swap the two lines and the order flips.

C++

```
void nToOne(int n){
    if(n == 0) return;
    cout << n << " ";    // print first
    nToOne(n - 1);        // then recurse
}
```

PYTHON

```
def n_to_one(n):
    if n == 0:
        return
    print(n, end=" ")    # print first
    n_to_one(n - 1)      # then recurse
```

Complexity & notes: $O(n)$. Q22 vs Q23 teach the single most important recursion idea: work-before-call vs work-after-call.

Q24. Sum of First N Numbers

Easy

■ **Approach:** $\text{sum}(n) = n + \text{sum}(n-1)$, with $\text{sum}(0) = 0$. The function returns a value that the caller adds to n . (Closed form $n(n+1)/2$ also works, but the recursion is the lesson.)

C++

```
int sumN(int n){
    if(n == 0) return 0;    // base case
    return n + sumN(n - 1); // combine
}
```

PYTHON

```
def sum_n(n):
    if n == 0:                # base case
        return 0
    return n + sum_n(n - 1) # combine
```

Complexity & notes: $O(n)$. This is 'functional' recursion — each call RETURNS a value that gets combined, unlike the void print ones.

Q25. Factorial of N

Easy

■ **Approach:** $n! = n * (n-1)!$, with $0! = 1$. Same return-and-combine shape as sum, but multiplying.

C++

```
long factorial(int n){
    if(n == 0) return 1;    // base case
    return n * factorial(n-1);
}
```

PYTHON

```
def factorial(n):
    if n == 0:                # base case
        return 1
    return n * factorial(n-1)
```

Complexity & notes: $O(n)$. Use long/long long in C++ — factorials overflow int fast (13! already exceeds int).

Q26. Reverse an Array (recursion)

Easy

■ **Approach:** Two pointers l and r from the ends. Swap them, then recurse inward ($l+1, r-1$). Base case: $l \geq r$ (pointers met/crossed).

C++

```
void reverse(vector<int>& a, int l, int r){
    if(l >= r) return;    // base case
    swap(a[l], a[r]);    // swap ends
    reverse(a, l+1, r-1); // move inward
}
```

PYTHON

```
def reverse(a, l, r):
    if l >= r:                # base case
        return
    a[l], a[r] = a[r], a[l] # swap ends
    reverse(a, l+1, r-1)    # move inward
```

Complexity & notes: $O(n)$ time, $O(n)$ stack. The two-pointer-from-ends idea returns constantly (palindromes, two-sum, etc.).

Q27. Check Palindrome String (recursion)

Easy

■ **Approach:** Compare the outer characters (l and r). If they differ, it's not a palindrome. If they match, recurse inward. Base case $l \geq r$ means all pairs matched.

C++

```
bool isPal(string& s, int l, int r){
    if(l >= r) return true;    // all matched
    if(s[l] != s[r]) return false;
    return isPal(s, l+1, r-1);
}
```

PYTHON

```
def is_pal(s, l, r):
    if l >= r:                # all matched
        return True
    if s[l] != s[r]:
        return False
    return is_pal(s, l+1, r-1)
```

Complexity & notes: $O(n)$. Same two-pointer skeleton as Q26, but comparing instead of swapping.

Q28. Fibonacci Number (recursion)

Easy

■ **Approach:** $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$, base cases $\text{fib}(0)=0$, $\text{fib}(1)=1$. Note TWO recursive calls — this makes the naive version exponential (repeated subproblems). Memoize to fix.

C++

```
int fib(int n){
    if(n < 2) return n;        // fib(0)=0, fib(1)=1
    return fib(n-1) + fib(n-2);
}
```

PYTHON

```
def fib(n):
    if n < 2:                  # base cases
        return n
    return fib(n-1) + fib(n-2)
```

Complexity & notes: Naive: $O(2^n)$ — the same subproblems recompute. Memoization (cache results) drops it to $O(n)$. This is your gateway to dynamic programming.

Hashing (Q29–Q31)

Q29. Hashing — Theory & Precompute

Easy

■ **Approach:** Hashing = trade memory for speed. Instead of scanning the array for every query, PRECOMPUTE a frequency table once ($O(n)$); then each 'how many times does x appear?' query is $O(1)$. Use an array for small bounded values, a hash map otherwise.

C++

```
// precompute frequency, then O(1) queries
unordered_map<int,int> mp;
for(int x : a) mp[x]++;      // O(n) once
cout << mp[5];              // O(1) per query
```

PYTHON

```
from collections import Counter
cnt = Counter(a)             # O(n) once
print(cnt[5])                # O(1) per query
```

Complexity & notes: Build once $O(n)$, then $O(1)$ average per lookup. Hash map worst case is $O(n)$ under heavy collisions, but effectively $O(1)$. This idea powers a huge fraction of later problems.

Q30. Count Frequency of Elements

Easy

■ **Approach:** Walk the array once, incrementing a counter per value in a hash map. The map's key is the element, the value is its count.

C++

```
map<int,int> freq(vector<int>& a){
    map<int,int> m;
    for(int x : a) m[x]++;    // count each
    return m;
}
```

PYTHON

```
from collections import Counter
def freq(a):
    return dict(Counter(a))  # element -> count
```

Complexity & notes: $O(n)$. `unordered_map` is $O(1)$ average (use when order doesn't matter); `map` is ordered but $O(\log n)$ per op.

Q31. Highest / Lowest Frequency Element

Easy

■ **Approach:** Build the frequency map (Q30), then scan its entries once tracking the key with the max count and the key with the min count.

C++

```
pair<int,int> hiLo(vector<int>& a){
    map<int,int> c; for(int x:a) c[x]++;
    int hi=a[0], lo=a[0];
    for(auto& [k,v] : c){
        if(v > c[hi]) hi = k;    // most frequent
        if(v < c[lo]) lo = k;    // least frequent
    }
    return {hi, lo};
}
```

PYTHON

```
from collections import Counter
def hi_lo(a):
    c = Counter(a)
    hi = max(c, key=lambda k: c[k])
    lo = min(c, key=lambda k: c[k])
    return hi, lo
```

Complexity & notes: $O(n)$ to build + $O(u)$ to scan (u = unique values). max/min with a key= is the Pythonic way to argmax over a dict.

Sorting Algorithms (Q32–Q38)

Q32. Selection Sort

Easy

■ **Approach:** For each position i , find the MINIMUM in the unsorted right part and swap it into place. The left part grows sorted. Exactly $n-1$ swaps (fewest of the elementary sorts), but always $O(n^2)$ comparisons.

C++

```
void selection(vector<int>& a){
    int n = a.size();
    for(int i = 0; i < n-1; i++){
        int m = i;
        for(int j = i+1; j < n; j++){
            if(a[j] < a[m]) m = j;    // find min
        }
        swap(a[i], a[m]);           // place it
    }
}
```

PYTHON

```
def selection(a):
    n = len(a)
    for i in range(n-1):
        m = i
        for j in range(i+1, n):
            if a[j] < a[m]: m = j    # find min
        a[i], a[m] = a[m], a[i]     # place it
    return a
```

Complexity & notes: $O(n^2)$ all cases. In-place, NOT stable, not adaptive. Minimizes writes ($n-1$ swaps).

Q33. Bubble Sort

Easy

■ **Approach:** Repeatedly compare ADJACENT pairs and swap if out of order; the largest 'bubbles' to the end each pass. The swapped flag gives an $O(n)$ best case on already-sorted input.

C++

```
void bubble(vector<int>& a){
    int n = a.size();
    for(int i = 0; i < n-1; i++){
        bool sw = false;
        for(int j = 0; j < n-1-i; j++){
            if(a[j] > a[j+1]){
                swap(a[j], a[j+1]); sw = true;
            }
        }
        if(!sw) break;        // already sorted
    }
}
```

PYTHON

```
def bubble(a):
    n = len(a)
    for i in range(n-1):
        sw = False
        for j in range(n-1-i):
            if a[j] > a[j+1]:
                a[j],a[j+1] = a[j+1],a[j]; sw = True
        if not sw: break      # already sorted
    return a
```

Complexity & notes: Best $O(n)$ (sorted), avg/worst $O(n^2)$. In-place, stable, adaptive (thanks to the swapped flag).

Q34. Insertion Sort

Easy

■ **Approach:** Grow a sorted left region; take each new element as a 'key' and slide it left past all bigger elements into its spot. Best of the elementary sorts: stable, adaptive, great on small/nearly-sorted arrays.

C++

```
void insertion(vector<int>& a){
    for(int i = 1; i < (int)a.size(); i++){
        int key = a[i], j = i - 1;
        while(j >= 0 && a[j] > key){
            a[j+1] = a[j]; j--; // shift right
        }
        a[j+1] = key; // drop key
    }
}
```

PYTHON

```
def insertion(a):
    for i in range(1, len(a)):
        key = a[i]; j = i - 1
        while j >= 0 and a[j] > key:
            a[j+1] = a[j]; j -= 1 # shift right
        a[j+1] = key # drop key
    return a
```

Complexity & notes: Best $O(n)$ (nearly sorted), avg/worst $O(n^2)$. In-place, stable, adaptive. Used by real sorts for small subarrays.

Q35. Merge Sort

Medium

■ **Approach:** Divide and conquer: split the array in half, recursively sort each half, then MERGE the two sorted halves into one. The merge walks both halves with two pointers, always taking the smaller front element.

C++

```
void mergeSort(vector<int>& a, int l, int r){
    if(l >= r) return;          // base: 0/1 elem
    int m = (l + r) / 2;
    mergeSort(a, l, m);        // sort left
    mergeSort(a, m+1, r);      // sort right
    vector<int> t; int i=l, j=m+1;
    while(i<=m && j<=r)        // merge
        t.push_back(a[i]<=a[j] ? a[i++] : a[j++]);
    while(i<=m) t.push_back(a[i++]);
    while(j<=r) t.push_back(a[j++]);
    for(int k=0;k<(int)t.size();k++) a[l+k]=t[k];
}
```

PYTHON

```
def merge_sort(a):
    if len(a) <= 1: return a    # base case
    m = len(a)//2
    L = merge_sort(a[:m])      # sort left
    R = merge_sort(a[m:])      # sort right
    res=[]; i=j=0
    while i<len(L) and j<len(R): # merge
        if L[i]<=R[j]: res.append(L[i]); i+=1
        else: res.append(R[j]); j+=1
    return res + L[i:] + R[j:]
```

Complexity & notes: $O(n \log n)$ ALL cases — reliable. Stable. Uses $O(n)$ extra space (out-of-place). The go-to when you need guaranteed $n \log n$ or stability.

Q36. Recursive Bubble Sort

Easy

■ **Approach:** Bubble sort framed recursively: one pass bubbles the largest to the end, then recurse on the first $n-1$ elements. Base case: $n == 1$ (single element is sorted).

C++

```
void recBubble(vector<int>& a, int n){
    if(n == 1) return;           // base case
    for(int j = 0; j < n-1; j++) // one pass
        if(a[j] > a[j+1]) swap(a[j], a[j+1]);
    recBubble(a, n - 1);         // shrink
}
```

PYTHON

```
def rec_bubble(a, n):
    if n == 1:                    # base case
        return a
    for j in range(n-1):         # one pass
        if a[j] > a[j+1]:
            a[j],a[j+1] = a[j+1],a[j]
    return rec_bubble(a, n-1)    # shrink
```

Complexity & notes: $O(n^2)$. Same behaviour as iterative bubble — this version just teaches how a loop maps to recursion.

Q37. Recursive Insertion Sort

Easy

■ **Approach:** Insertion sort framed recursively: assume the first i elements are sorted, insert element i into them, then recurse for $i+1$. Base case: i reaches the array end.

C++

```
void recInsertion(vector<int>& a, int i){
    if(i >= (int)a.size()) return;    // base
    int key = a[i], j = i - 1;
    while(j >= 0 && a[j] > key){
        a[j+1] = a[j]; j--;
    }
    a[j+1] = key;
    recInsertion(a, i + 1);           // next
}
```

PYTHON

```
def rec_insertion(a, i=1):
    if i >= len(a):                    # base
        return a
    key = a[i]; j = i - 1
    while j >= 0 and a[j] > key:
        a[j+1] = a[j]; j -= 1
    a[j+1] = key
    return rec_insertion(a, i+1)       # next
```

Complexity & notes: $O(n^2)$. The recursion replaces the outer for-loop; the inner shifting while-loop stays the same.

Q38. Quick Sort

Medium

■ **Approach:** Pick a pivot (here the last element), PARTITION so smaller elements go left and larger go right, then recurse on both sides. Sorts in place. Average $O(n \log n)$; worst $O(n^2)$ on bad pivots (mitigate with a random pivot).

C++

```
int partition(vector<int>& a, int lo, int hi){
    int p = a[hi], i = lo;
    for(int j = lo; j < hi; j++)
        if(a[j] < p) swap(a[i++], a[j]);
    swap(a[i], a[hi]); // pivot in place
    return i;
}
void quick(vector<int>& a, int lo, int hi){
    if(lo >= hi) return;
    int p = partition(a, lo, hi);
    quick(a, lo, p-1); quick(a, p+1, hi);
}
```

PYTHON

```
def quick(a, lo, hi):
    if lo >= hi: return
    p = a[hi]; i = lo
    for j in range(lo, hi):
        if a[j] < p:
            a[i], a[j] = a[j], a[i]; i += 1
    a[i], a[hi] = a[hi], a[i] # pivot placed
    quick(a, lo, i-1); quick(a, i+1, hi)
```

Complexity & notes: Avg $O(n \log n)$, worst $O(n^2)$. In-place ($O(\log n)$ stack), NOT stable. Usually fastest in practice due to cache-friendliness.

Arrays — First Problems (Q39–Q40)

Q39. Largest Element in Array

Easy

■ **Approach:** Track a running maximum in a single pass: start with the first element, and update whenever you see something bigger. No sorting needed.

C++

```
int largest(vector<int>& a){
    int mx = a[0];
    for(int x : a)
        if(x > mx) mx = x;    // update running max
    return mx;
}
```

PYTHON

```
def largest(a):
    mx = a[0]
    for x in a:
        if x > mx: mx = x    # update running max
    return mx
```

Complexity & notes: $O(n)$ time, $O(1)$ space. Beats sorting ($O(n \log n)$). 'Track the best so far in one pass' is a core pattern.

Q40. Second Largest Element

Easy

■ **Approach:** Do it in ONE pass with two variables: the largest and the second largest so far. For each value: if it beats first, first becomes second and the value becomes first; else if it's between second and first, update second. (Sorting works too but is $O(n \log n)$.)

C++

```
int secondLargest(vector<int>& a){
    long f = LONG_MIN, s = LONG_MIN;
    for(int x : a){
        if(x > f){ s = f; f = x; }    // new top
        else if(x < f && x > s) s = x; // new 2nd
    }
    return s;
}
```

PYTHON

```
def second_largest(a):
    first = second = float("-inf")
    for x in a:
        if x > first:
            second = first; first = x    # new top
        elif first > x > second:
            second = x                  # new 2nd
    return second
```

Complexity & notes: $O(n)$ time, $O(1)$ space. The $x < f$ check avoids duplicates of the max wrongly becoming 'second'. Classic single-pass tracking.

That's Q21–40 in both languages. The big patterns you just built: **recursion** (base case + smaller call, and the crucial work-before vs work-after-call ordering), **hashing** (precompute once for $O(1)$ lookups), the **sort family** (know each one's stability / adaptivity / complexity cold — a guaranteed interview question), and **single-pass tracking** for array extremes. Re-type each from a blank page in both C++ and Python. Next up: the rest of Step 3 (Arrays) — the real problem-solving begins.